

[Det här är ett lösningsförslag till katan med romerska nummer och inget facit. Lösningarna kan självklart se ut på en mängd olika sätt, men genom att arbeta inkrementellt bör man i alla fall stöta på samma hinder.]

Testfall: 1-3

Börja med att få testen att bli gröna med minsta möjliga ansträngning:

```
public static String convert(int number)
{
    if (number == 1) { return "I"; }
    else if (number == 2) { return "II"; }
    else { return "III"; }
}
```

Refaktorering efter testfall 3:

Vi ser nu ett hyfsat uppenbart mönster. Vi kan generalisera koden med en for-loop i stället som bygger på en sträng åt oss:

```
public static String convert(int number)
{
    String result = "";

    for (int i = 0; i < number; i++)
    {
        result += "I";
    }

    return result;
}
```

Testfall 4:

Nu spricker loopen. Vi fortsätter lösa det här undantagsfallet på enklast möjliga vis just nu:

```
public static String convert(int number)
{
    if (number == 4) { return "IV"; }
    String result = "";

    for (int i = 0; i < number; i++)
    {
        result += "I";
    }

    return result;
}
```

Testfall 5:

Vi löser det på samma vis som 4 just nu:

```
public static String convert(int number)
{
    if (number == 4) { return "IV"; }
    if (number == 5) { return "V"; }
    String result = "";

    for (int i = 0; i < number; i++)
    {
        result += "I";
    }

    return result;
}
```

Testfall 6-8:

Här inser vi snart att logiken för att skriva ut 6, 7 och 8 är samma som för 1, 2 och 3. Om numret är högre än fem kan vi alltså börja strängen med V, dra ifrån 5 från numret, och sedan fortsätta skriva ut ettorna efter. Det här kommer funka för tal 6-8. Vi låter IV vara ett särskilt undantagsfall så länge:

```
public static String convert(int number)
{
    if (number == 4) { return "IV";}

    String result = "";

    if (number >= 5)
    {
        result += "V";
        number -= 5;
    }

    for (int i = 0; i < number; i++)
    {
        result += "I";
    }

    return result;
}
```

Testfall 9:

9 bryter mönstret igen!

Tänk på att arbeta i små inkrementella steg: vi behöver inte tänka ut krånglig logik för att generalisera det här än. Vi gör ett speciellt undantag i stället för tillfället, precis som vi gjorde för IV:

```
public static String convert(int number)
{
    if (number == 4) { return "IV";}
    if (number == 9) { return "IX";}

    String result = "";

    if (number >= 5)
    {
        result += "V";
        number -= 5;
    }

    for (int i = 0; i < number; i++)
    {
        result += "I";
    }

    return result;
}
```

Testfall 10-13:

Vi kan lätt återanvända lösningen för 5-8 här utan vidare ansträngning:

```
public static String convert(int number)
{
    if (number == 4) { return "IV"; }
    if (number == 9) { return "IX"; }

    String result = "";

    if (number >= 10)
    {
        result += "X";
        number -= 10;
    }

    if (number >= 5)
    {
        result += "V";
        number -= 5;
    }

    for (int i = 0; i < number; i++)
    {
        result += "I";
    }

    return result;
}
```

Testfall 14:

Här kan vi stanna upp en stund och fundera på hur vår kod är strukturerad. Vi har redan ett speciellt undantag för 4, men kan vi inte bara flytta det i stället så att det appenderas till strängen efter if-checkarna för 10 och 5? Om talet bara är 4 kommer IV att stoppas in i strängen och sen körs inte for-loopen. Om talet är 14 kommer vi redan ha stoppat in ett X före IV! Vi kan alltså lösa det här problemet genom att bara strukturera om befintlig kod:

```
public static String convert(int number)
{
    if (number == 9) { return "IX";}

    String result = "";

    if (number >= 10)
    {
        result += "X";
        number -= 10;
    }

    if (number >= 5)
    {
        result += "V";
        number -= 5;
    }

    if (number == 4)
    {
        result += "IV";
        return result;
    }

    for (int i = 0; i < number; i++)
    {
        result += "I";
    }

    return result;
}
```

Testfall 15-18:

Dessa löser sig självt nu! XV, XVI, XVII och XVIII hanteras redan av algoritmen!

Testfall 19:

Här kan vi stanna upp igen och titta tillbaka på föregående lösning: vi kan ju hantera 9 på exakt samma vis som 4! Allt som krävs är en lätt omskrivning av existerande kod:

```
public static String convert(int number)
{
    String result = "";

    if (number >= 10)
    {
        result += "X";
        number -= 10;
    }

    if (number == 9)
    {
        result += "IX";
        return result;
    }

    if (number >= 5)
    {
        result += "V";
        number -= 5;
    }

    if (number == 4)
    {
        result += "IV";
        return result;
    }

    for (int i = 0; i < number; i++)
    {
        result += "I";
    }
}
```

```
}  
  
    return result;  
}
```

Testfall 20:

Det här kommer inte fungera: programmet ger oss XVV i stället för XX. Men vi kan stanna upp och fundera över varför, för lösningen är ganska uppenbar om vi funderar litegrann. Allt som krävs är att vi ändrar fem tecken!

Kolla på villkoret för tio:

```
if (number >= 10)  
{  
    result += "X";  
    number -= 10;  
}
```

Här använder vi ju if, vilket innebär att vi går vidare om vi dragit från 10, men vi vill ju FORTSÄTTA dra ifrån 10 tills vi inte kan subtrahera fler tior (för 20 är ju 10 + 10).

Allt vi behöver ändra är alltså if-villkoret till en while:

```
while (number >= 10)  
{  
    result += "X";  
    number -= 10;  
}
```

Nu fungerar testet igen! Och eftersom det fungerade för 1-9 och 11-20 kommer det nu fungera för 21-30 och 30-39! En algoritm börjar nu sakta träda fram. Vi känner historiens vingslag piska oss i ansiktet.

Testfall 40:

Programmet kommer att påstå att 40 är XXXX, men vi vet att det ska vara XL. Igen, låt oss lösa detta på minsta möjliga vis:

```
public static String convert(int number)
{
    String result = "";

    if (number >= 40)
    {
        result += "XL";
        number -= 40;
    }

    while (number >= 10)
    {
        result += "X";
        number -= 10;
    }

    if (number == 9)
    {
        result += "IX";
        return result;
    }

    if (number >= 5)
    {
        result += "V";
        number -= 5;
    }

    if (number == 4)
    {
        result += "IV";
        return result;
    }

    for (int i = 0; i < number; i++)
    {
        result += "I";
    }
}
```

```

    }

    return result;
}

```

40-49 kommer nu också att fungera!

Testfall 50:

Programmet kommer ge oss XLX i stället för L nu, och om vi tittar på koden vi skrivit kan vi ana varför. Mönstret hittills är att de största talen måste avhandlas först. Problemet är alltså att vi kollar 40 innan 50.

Lös detta på minsta möjliga vis och lägg helt enkelt till en check för 50 före 40:

```

if (number >= 50)
{
    result += "L";
    number -= 50;
}

```

Koden fungerar, testet kommer att visa grönt - och vi har nu hanterat alla testfall ända upp till och med 89 eftersom logiken är densamma i övrigt!

Testfall 90:

90 kommer att skrivas ut som LXL, men det här är inte rätt. 100 är C, och vi vill ju ha 100 - 10; alltså XC!

Vi lägger till ytterligare en branching för 100 - och nu inser vi att programmet börjar bli ganska repetitivt:

```

public static String convert(int number)
{
    String result = "";

    if (number >= 90)
    {

```

```
    result += "XC";
    number -= 90;
}

if (number >= 50)
{
    result += "L";
    number -= 50;
}

if (number >= 40)
{
    result += "XL";
    number -= 40;
}

while (number >= 10)
{
    result += "X";
    number -= 10;
}

if (number == 9)
{
    result += "IX";
    return result;
}

if (number >= 5)
{
    result += "V";
    number -= 5;
}

if (number == 4)
{
    result += "IV";
    return result;
}
```

```

    for (int i = 0; i < number; i++)
    {
        result += "I";
    }

    return result;
}

```

Testfall 100:

100 kommer att generera ett rött test igen. Vi vet att det borde vara C men programmet kommer att skriva ut XCX. Vi **KAN** lägga till ännu en if-check och få testet att passera:

```

if (number >= 100)
{
    result += "C";
    number -= 100;
}

```

Refaktorering efter testfall 100:

...men det börjar bli ganska tjatigt. Mönstret i koden är nu ganska uppenbart, och kommer att fortsätta upprepas så här hela vägen upp till 1000. Det är dags att fundera på om vi kan generalisera koden lite i stället för att bara upprepa samma saker hela tiden! Hur får vi bort alla dessa if-checkar?

Kolla på algoritmen vi har skapat:

```

if (number >= 100)...lägg till C
if (number >= 90)... lägg till XC
if (number >= 50)... lägg till L
if (number >= 40)... lägg till XL
while (number >= 10)...X
if (number == 9)... lägg till IX
if (number >= 5)... lägg till V
if (number == 4)... lägg till IV
while (number >= 1)... lägg till I

```

Det här är samma struktur hela vägen - allt som ändras är siffror och symboler, men logiken är densamma. Vad vi har är detta:

100	C
90	XC
50	L
40	XL
10	X
9	IX
5	V
4	IV
1	I

Det här är bara en lista! Kan vi inte spara dessa värden någonstans och sen bara iterera genom dem i stället?

Vi har flera val just nu: vi skulle kunna skapa objekt och slänga i en ArrayList, eller använda en TreeMap, men den **enklaste lösningen** med så lite kod som möjligt är den vi är ute efter för tillfället (TDD varnar oss för att ta för stora kliv). Och de enklaste samlingarna vi har är arrayer.

Låt oss skapa två stycken arrayer, en för decimalvärden och en för romerska symboler:

```
int[] values = { 100, 90, 50, 40, 10, 9, 5, 4, 1 };
```

```
String[] romanNumerals = { "C", "XC", "L", "XL", "X", "IX", "V",  
"IV", "I" };
```

Låt oss sedan gå genom samlingen med värden, och fortsätta skriva ut och subtrahera det största värdet så länge vi kan, innan vi sedan fortsätter till nästa:

```
for (int i = 0; i < values.length; i++)  
{  
    while (number >= values[i])  
    {
```

```

        result += romanNumerals[i];
        number -= values[i];
    }
}

```

Vi har nu en algoritm som ser ut så här:

```

public static String convert(int number)
{
    int[] values = { 100, 90, 50, 40, 10, 9, 5, 4, 1 };
    String[] romanNumerals = { "C", "XC", "L", "XL", "X", "IX", "V",
                                "IV", "I" };

    String result = "";
    for (int i = 0; i < values.length; i++)
    {
        while (number >= values[i])
        {
            result += romanNumerals[i];
            number -= values[i];
        }
    }
    return result;
}

```

Kör testen igen. Allihop kommer fortfarande vara gröna. Vi har **ändrat strukturen för algoritmen, men inte beteendet**. Gör en kullerbytta av glädje!

Testfall 400, 500, 900 och 1000 kan hanteras på samma sätt: lägg till dem i arrayerna bara. Vi har nu en kort, generaliserad och kraftfull algoritm som gör exakt vad den ska!

Bonus - ytterligare refaktorering:

NU kan vi börja snygga till koden när vi har benat ut logiken. Vi hanterar just nu separata arrayer, vilket är fine, men det är inte särskilt robust. Vi är objektorienterade programmerare och vi gillar objektorienterade lösningar - låt oss skapa en dataklass som heter RomanNumber:

```
public class RomanNumber
{
    private final int decimalValue;
    private final String romanNumeral;

    public RomanNumber(int decimalValue, String romanNumeral)
    {
        this.decimalValue = decimalValue;
        this.romanNumeral = romanNumeral;
    }

    public int getDecimalValue() { return decimalValue; }
    public String getRomanNumeral() { return romanNumeral; }
}
```

Vi kan skapa en lista med sådana här objekt i stället nu, och slipper därmed separata arrayer:

```
ArrayList<RomanNumber> list = new ArrayList<>();
list.add(new RomanNumber(1000, "M"));
list.add(new RomanNumber(900, "CM"));
list.add(new RomanNumber(500, "D"));
list.add(new RomanNumber(400, "CD"));
list.add(new RomanNumber(100, "C"));
list.add(new RomanNumber(90, "XC"));
list.add(new RomanNumber(50, "L"));
list.add(new RomanNumber(40, "XL"));
list.add(new RomanNumber(10, "X"));
list.add(new RomanNumber(9, "IX"));
list.add(new RomanNumber(5, "V"));
list.add(new RomanNumber(4, "IV"));
list.add(new RomanNumber(1, "I"));
```

Allt vi behöver är att loopa genom dem nu:

```

String result = "";
for (RomanNumber romanNumber : list)
{
    while (number >= romanNumber.getDecimalValue())
    {
        result += romanNumber.getRomanNumeral();
        number -= romanNumber.getDecimalValue();
    }
}

```

Fullständig objektorienterad lösning:

```

public static String convert(int number)
{
    ArrayList<RomanNumber> list = new ArrayList<>();
    list.add(new RomanNumber(1000, "M"));
    list.add(new RomanNumber(900, "CM"));
    list.add(new RomanNumber(500, "D"));
    list.add(new RomanNumber(400, "CD"));
    list.add(new RomanNumber(100, "C"));
    list.add(new RomanNumber(90, "XC"));
    list.add(new RomanNumber(50, "L"));
    list.add(new RomanNumber(40, "XL"));
    list.add(new RomanNumber(10, "X"));
    list.add(new RomanNumber(9, "IX"));
    list.add(new RomanNumber(5, "V"));
    list.add(new RomanNumber(4, "IV"));
    list.add(new RomanNumber(1, "I"));

    String result = "";
    for (RomanNumber romanNumber : list)
    {
        while (number >= romanNumber.getDecimalValue())
        {
            result += romanNumber.getRomanNumeral();
            number -= romanNumber.getDecimalValue();
        }
    }
    return result;
}

```